

FACULTAD DE INGENIERÍA
LICENCIATURA EN INFORMÁTICA Y SISTEMAS
MANEJO E IMPLEMENTACIÓN DE ARCHIVOS

ESTUDIANTE: **CARNET:**

Juan Daniel Ulin Velazquez **1559325**

Lunes, 9 de Noviembre de 2026
QUETZALTENANGO, GUATEMALA

ESTRUCTURA DE LOS REGISTROS

En este laboratorio se hizo una agenda de contactos usando Python y una ventana sencilla con Tkinter. Los contactos se guardan en el archivo **contactos.dat**, y cada uno ocupa el mismo espacio para que se pueda llegar directamente a su posición.

Los campos son: ID (6 bytes), nombre (30 bytes), teléfono (15 bytes), correo (40 bytes) y ciudad (25 bytes). Se suma también 1 byte para el salto de línea. Por eso, **cada registro ocupa 117 bytes**.

Para guardar los datos se usa la función **convertir()**. Primero los textos se pasan a UTF-8 y después se utiliza **ljust()** para completar con espacios lo que falta. Si un dato se pasa del tamaño permitido, el programa muestra un error.

Fragmento del código:

```
CAMPOS = ["id", "nombre", "telefono", "correo", "ciudad"]
LARGOS = [6, 30, 15, 40, 25]
TAMANO = sum(LARGOS) + 1

def convertir(contacto):
    partes = []
    for campo, largo in zip(CAMPOS, LARGOS):
        texto = str(contacto[campo]).strip()
        dato = texto.encode("utf-8")
        if len(dato) > largo:
            raise ValueError("El campo " + campo + " es demasiado largo.")
        partes.append(dato.ljust(largo, b" "))
    return b"".join(partes) + b"\n"
```

ACCESO ALEATORIO CON SEEK Y TELL

La parte principal de este laboratorio es que no se necesita leer todo el archivo para buscar un contacto. Como los registros tienen tamaño fijo, se multiplica el número de registro por 117 para conocer el byte donde empieza.

Por ejemplo, el primer contacto empieza en el byte 0, el segundo en el 117 y el tercero en el 234. Con **seek()** se mueve el cursor a esa posición y con **tell()** se comprueba dónde quedó. Después solamente se leen los 117 bytes del registro.

Fragmento del código de lectura:

```
def leer(numero):
    with open(DATOS, "rb") as archivo:
        archivo.seek(numero * TAMANO)
        posicion = archivo.tell()
        contacto = separar(archivo.read(TAMANO))
    return contacto, posicion
```

Modificación del contacto

Al modificar un contacto se utiliza el modo **r+b** para leer y escribir en el mismo archivo. Se busca su posición y se sobreescribe únicamente ese registro. Los demás contactos no se cambian y el tamaño del archivo sigue siendo el mismo.

```
def modificar(contacto):
    numero_id = contacto["id"]
    buscar(numero_id)
    posicion = indice[str(numero_id)] * TAMANO
    bloque = convertir(contacto)
    with open(DATOS, "r+b") as archivo:
        archivo.seek(posicion)
        posicion = archivo.tell()
        archivo.write(bloque)
    guardar_hash()
    return posicion
```

ÍNDICE PRIMARIO EN JSON

Se utilizó un archivo llamado **indice.json** para guardar el ID de cada contacto junto con el número de registro donde se encuentra. No guarda los demás datos del contacto, solamente el ID y su posición.

El índice de los tres contactos de ejemplo es el siguiente:

```
{
    "101": 0,
    "102": 1,
    "103": 2
}
```

Cuando se hace una búsqueda, primero se revisa si el ID está en el diccionario del índice. Si existe, se obtiene su número de registro y se llama a **leer()**, que utiliza seek(). El índice se actualiza cada vez que se agrega un contacto nuevo.

Fragmento del código de búsqueda:

```
def buscar(numero_id):
    if str(numero_id) not in indice:
        raise ValueError("No existe un contacto con ese ID.")
    contacto, posicion = leer(indice[str(numero_id)])
    if contacto["id"] != numero_id:
        raise ValueError("El índice no coincide con el archivo.")
    return contacto, posicion
```

Este método es más rápido para buscar un ID porque evita ir revisando contacto por contacto. En una búsqueda secuencial tendríamos que leer registros hasta encontrar el que necesitamos.

EXPORTACIÓN A JSON Y PDF

La opción de exportar a JSON crea el archivo **contactos.json** con todos los datos de los contactos. Para hacerlo se usa la función todos(), que lee los registros, y después **json.dump()** para guardarlos de una forma ordenada.

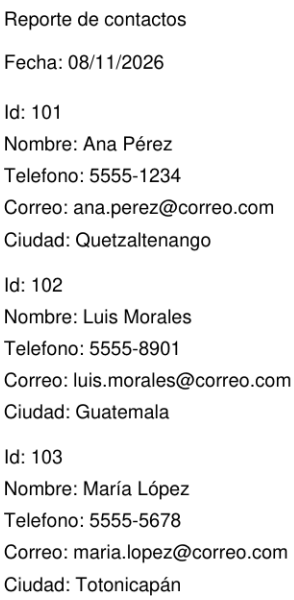
También se puede generar un reporte PDF. Primero el programa exporta la información a JSON; después lee ese JSON con **json.load()** y utiliza la librería **fpdf2** para crear el archivo **reporte.pdf** con los contactos y la fecha.

Fragmento del código:

```
def exportar_json():
    with open(EXPORTADO, "w", encoding="utf-8") as archivo:
        json.dump(todos(), archivo, indent=4, ensure_ascii=False)

def exportar_pdf():
    from fpdf import FPDF
    exportar_json()
    with open(EXPORTADO, "r", encoding="utf-8") as archivo:
        contactos = json.load(archivo)
```

Ejemplo del reporte generado:



Reporte de contactos

Fecha: 08/11/2026

Id: 101
Nombre: Ana Pérez
Telefono: 5555-1234
Correo: ana.perez@correo.com
Ciudad: Quetzaltenango

Id: 102
Nombre: Luis Morales
Telefono: 5555-8901
Correo: luis.morales@correo.com
Ciudad: Guatemala

Id: 103
Nombre: María López
Telefono: 5555-5678
Correo: maria.lopez@correo.com
Ciudad: Totonicapán

VERIFICACIÓN DE INTEGRIDAD

Se utilizó **SHA-256** para detectar si alguien cambia el archivo de contactos fuera de la aplicación. Cada vez que se agrega o modifica un contacto, se calcula un nuevo hash de contactos.dat y se guarda en **contactos.sha256**.

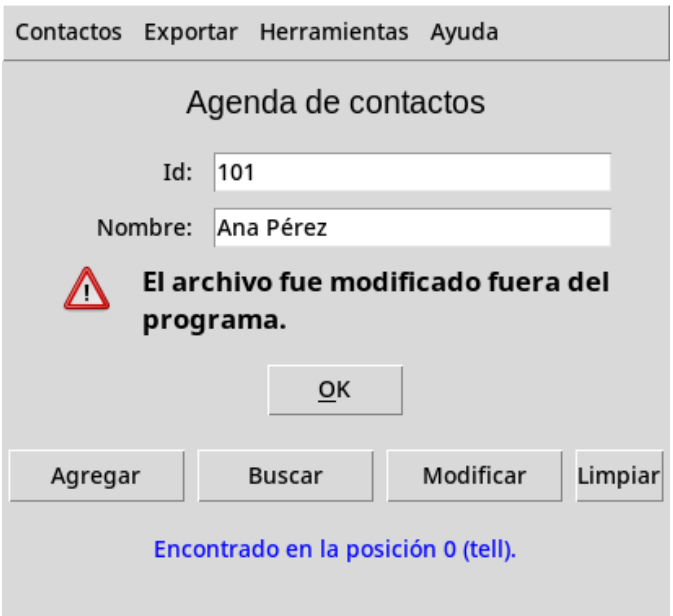
Al iniciar la aplicación y al usar la opción Verificar integridad, el programa compara el hash actual con el que estaba guardado. Si son diferentes muestra una advertencia. Esto permite detectar cambios, aunque no impide que otra persona modifique el archivo.

Fragmento del código:

```
def calcular_hash():
    with open(DATOS, "rb") as archivo:
        return hashlib.sha256(archivo.read()).hexdigest()

def comprobar_hash():
    if not os.path.exists(HASH):
        return False
    with open(HASH, "r", encoding="utf-8") as archivo:
        anterior = archivo.read().strip()
    return anterior == calcular_hash()
```

Evidencia de un archivo alterado manualmente:



RESPALDO ZIP

Para hacer el respaldo se utilizó la librería **zipfile**. El programa crea un archivo llamado **respaldo.zip** que contiene contactos.dat e indice.json. De esta manera se pueden guardar juntos los datos y el índice.

Con los tres contactos del ejemplo, contactos.dat ocupa 351 bytes y el índice ocupa 44 bytes. En total son **395 bytes**. El respaldo ZIP ocupa **416 bytes**; en este caso es un poco más grande porque son pocos datos y el ZIP también guarda información interna.

Fragmento del código:

```
def respaldo():
    original = os.path.getsize(DATOS) + os.path.getsize(INDICE)
    with zipfile.ZipFile(RESPALDO, "w", zipfile.ZIP_DEFLATED) as comprimido:
        comprimido.write(DATOS, "contactos.dat")
        comprimido.write(INDICE, "indice.json")
    return original, os.path.getsize(RESPALDO)
```

Metadatos y permisos

En la opción Información del archivo se usa **os.stat()** para mostrar el tamaño y la última fecha de modificación. También se utiliza **os.access()** para revisar los permisos de lectura y escritura.

Manejo de errores

Se utilizaron bloques **try-except** para mostrar avisos cuando hay datos incorrectos o problemas con archivos y permisos. Si no existe el archivo de contactos, el programa lo crea vacío; si falta el índice o tiene un formato incorrecto, intenta reconstruirlo.

```
def ejecutar(funcion):
    try:
        funcion()
    except (OSError, ValueError, UnicodeError, ImportError) as error:
        messagebox.showerror("Error", str(error))
```

CAPTURAS DE LA APLICACIÓN

La interfaz cuenta con campos para el ID, nombre, teléfono, correo y ciudad. Tiene botones para agregar, buscar, modificar y limpiar. En el menú también están las opciones de exportación y herramientas.

Búsqueda de un contacto por ID:



Aviso al comprobar la integridad del archivo:

